

Audion Docs AI v3

Audion Docs AI v3 - портативный Windows-first пайплайн для извлечения документов, LLM-аудита, локальной DOCX-нормализации по готовому audit JSON, структурированных отчётов и опциональных DOCX-якорей для ручной проверки.

```
Scan DOCX/PPTX -> COM PDF/render-map export -> Audit -> Report -> Normalize /  
Annotate
```

Проект рассчитан на рабочие Windows-сценарии: DOCX/PPTX через Microsoft Office COM, повторяемое чанкирование, resume/cache для LLM-аудита и запуск через `.cmd`-лаунчеры от portable Python runtime.

Текущее состояние

Отдельных legacy audit-лаунчеров под отдельные пары провайдеров больше нет. OpenAI, Gemini, xAI и Anthropic Claude выбираются в GUI; размеры чанков, retry/timeout-настройки и имена prompt-файлов вынесены в `config\llm_settings.yaml`. Штатные модели приложения: `gpt-5.6-luna`, `gemini-3.6-flash`, `grok-4.3` и `claude-sonnet-5`; выбранная в GUI модель или ручной override имеет приоритет. GUI-списки, cache/избранное и явные smoke-проверки остаются локальными.

Видимый Project launcher теперь DOCX/PPTX-first. Он стартует от `input\` и работает напрямую через рекурсивный pipeline; старые действия подготовки Markdown в Project-меню больше не показываются.

Корневые лаунчеры:

- `builder_main.cmd` - сборка, установка, проверка окружения, release, запуск Project/Tools
- `launcher_gui.cmd` - desktop GUI-shell поверх рекурсивного pipeline
- `launcher_project.cmd` - английский workflow проекта, ставит `AUDION_REPORT_LANG=en`
- `launcher_project_ru.cmd` - русский workflow проекта, ставит `AUDION_REPORT_LANG=ru`
- `launcher_tools.cmd` - служебные инструменты, лицензии, GitHub cleanup

- `cleanup_project.cmd` - подготовка дерева исходников к публикации на GitHub

GUI-shell

GUI запускается через:

```
launcher_gui.cmd
```

Он не заменяет CLI, а оборачивает `system_core\pipeline.py`, `system_core\document_normalizer.py` и `system_core\doc_task_runner.py`: команды и параметры выбираются слева, статус и живой терминальный вывод остаются справа. Экраны OpenAI, Gemini, xAI и Claude группируют ключ и модель вместе, держат правила аудита ниже и дают одно действие **ЗАПУСТИТЬ**: audit-команда создаёт или переиспользует render-артефакты, вызывает LLM, проверяет язык результата и только затем пишет XLSX/DOCX-отчёты и annotated-копии. Корневые команды **ИСПРАВЛЕНИЕ OPENAI**, **ИСПРАВЛЕНИЕ GEMINI**, **ИСПРАВЛЕНИЕ XAI** и **ИСПРАВЛЕНИЕ CLAUDE** повторно не вызывают LLM: они читают готовые `logs\**\*_audit.json` своего провайдера и применяют только `fix_mode: safe_replace`, `confidence: high`, точные одиночные `old_text -> new_text` замены к DOCX-копиям. TASK-команды читают DOCX/PPTX/XLSX/PDF; PDF используется как read-only текстовые страницы для извлечения в отчёты, не для правки. Всё сомнительное остаётся в таблице **Unresolved Items** в DOCX-отчёте нормализации. Глубина каждого audit provider выбирается радиокнопками: OpenAI передаёт reasoning effort, Gemini 3 — настоящий **thinking_level**, xAI — настоящий **reasoning_effort**, Claude — уровень **effort** при постоянном adaptive thinking. Второстепенные поля размера чанков/overlap/min chunks, retry/timeout, ручной override модели, resume/cache и render-map strictness спрятаны в сворачиваемом блоке **Дополнительно**. Python запускается без буферизации, поэтому конфигурация и прогресс каждого чанка появляются в журнале сразу. Список моделей кэшируется локально в `config\gui_model_cache.json`; отдельная кнопка делает маленькую smoke-проверку выбранной модели и сохраняет статус доступа с датой. Пустой model fallback берёт последнюю ОК-проверенную модель, затем первую избранную модель, но никогда не выбирает произвольную строку из сырого provider list.

Кнопка **Обновить наименования** у моделей делает live-запрос к provider API с выбранным ключом текущего провайдера; **СБРОСИТЬ** очищает локальный модельный кэш этого провайдера. Resume/cache для LLM-аудита и TASK подписывается хэшем исходного документа и ключевыми параметрами запуска, поэтому отредактированный файл с тем же названием не должен брать старый LLM-ответ.

Канонические названия Workbench

Кнопки Workbench: `Источник`, `Добавить файл...`, `Назначение`, `Сбросить`, `Удалить`, `Список` (EN: `Source`, `Add file...`, `Target`, `Reset`, `Delete`, `List`). Выбранная папка или отдельный документ передаётся backend напрямую. `Сбросить` возвращает проектные `input\`/`output\` и не удаляет файлы; `Удалить` очищает выбранные Source/Target только после подтверждения.

GUI-only настройки:

- `config\gui_settings.yaml`
- `config\tool_manifest.yaml`

Внутренняя документация текущего GUI/workflow:

- `docs\GUI_WORKFLOW_RU.md`
- `docs\DOCUMENT_NORMALIZATION_RU.md`
- `docs\GITHUB_PREP_RU.md`

Основной workflow

Рекурсивный DOCX/PPTX pipeline

Новый рекурсивный pipeline находится в:

```
system_core\pipeline.py
```

Он читает только из:

```
input\
```

Он пишет:

- пользовательские файлы в `output\`
- JSON-логи и карты в `logs\`
- промежуточные render-артефакты в `work\`

Основные команды:

```
runtime\python.exe system_core\pipeline.py scan
runtime\python.exe system_core\pipeline.py render --recursive --renderer com
runtime\python.exe system_core\pipeline.py audit --recursive --renderer com
runtime\python.exe system_core\pipeline.py audit --recursive --renderer com --
require-render-map
runtime\python.exe system_core\pipeline.py report --from-logs logs --report-lang ru
runtime\python.exe system_core\pipeline.py annotate --from-logs logs
runtime\python.exe system_core\document_normalizer.py --provider openai --from-logs
logs
runtime\python.exe system_core\document_normalizer.py --provider gemini --from-logs
logs
```

COM-renderer создаёт marked OOXML-копии, экспортирует их через Microsoft Office, извлекает PDF-текст через PyMuPDF и строит render map. В strict-режиме pipeline падает, если render-map нельзя получить, а не придумывает страницы.

Видимые действия Project launcher:

- SCAN input
- COM-ЭКСПОРТ PDF / RENDER MAP
- AUDIT COM
- AUDIT COM STRICT
- REPORT FROM LOGS
- ANNOTATE FROM LOGS
- ИСПРАВЛЕНИЕ OPENAI
- ИСПРАВЛЕНИЕ GEMINI
- ИСПРАВЛЕНИЕ XAI
- ИСПРАВЛЕНИЕ CLAUDE
- ЗАПУСТИТЬ ВСЁ COM WORKFLOW
- группа OpenAI audit
- группа Gemini audit
- открыть input, output, logs, work, config

Нормализация документов по аудиту

Нормализация - отдельный локальный шаг после аудита. Она использует готовые

logs***__audit.json, выбирает записи нужного провайдера по meta.provider и создаёт

нормализованные DOCX-копии:

```
output\<relative_path>\<stem>__normalized.docx
```

Автоматически применяются только безопасные точные замены из audit JSON: `fix_mode:` `safe_replace`, `confidence: high`, `block_id`, `old_text` и `new_text`. Старые или неоднозначные записи уходят в таблицу `Unresolved Items` в отчёте нормализации. Отчёты пишутся в `output\_normalization\`, patch-plan - в `report\document_normalization\`.

Конфигурация

Главный файл LLM-настроек:

```
config\llm_settings.yaml
```

В YAML заданы application defaults: OpenAI `gpt-5.6-luna`, Gemini `gemini-3.6-flash`, xAI `grok-4.3`, Anthropic `claude-sonnet-5`. GUI-списки моделей, избранное и smoke-статусы живут в `config/gui_model_cache.json`; явный GUI-выбор или manual override важнее default.

Лаунчеры читают его через:

```
system_core\config_resolver.py
```

Prompt-файлы лежат в `config\`:

- `audit_system_prompt.md`
- `audit_user_prompt.md`
- `ocr_prompt.md`
- `resolver.md`

Audit prompt просит модель помечать только однозначные точные замены как `safe_replace`; спорные случаи остаются `requires_review` и не применяются машинно.

При `report_lang=ru` prompt требует русский язык человекочитаемых полей. Полностью или преимущественно английские `problem` и `recommendation` отправляются тому же provider на короткий language-repair. При неуспехе выполняется один fallback: OpenAI/xAI → `gemini-3.6-flash` (`minimal`), Gemini/Anthropic → `gpt-5.6-luna` (`low`). Готовые audit chunks не

повторяются. Обе попытки, диагностика и cache записываются в `logs\<stem>__language_repair.json`; при двойном браке XLSX/DOCX не создаются.

Файлы правил аудита лежат в `config\audit_rules\`. Активный файл для CLI/TUI и default GUI-запусков:

- `config\audit_rules\active_audit_rules.md`

Инструкции TASK для произвольных LLM-действий по документам лежат в `config\doc_tasks\`. Активная task-инструкция:

- `config\doc_tasks\active_doc_task.md`

Для разовых запросов GUI может передать inline-инструкцию без Markdown-файла. Такие быстрые инструкции кэшируются и добавляются в избранное в:

- `config\gui_doc_task_quick_cache.json`

Document task runner рекурсивно читает DOCX/PPTX/XLSX/PDF из `input\`. PDF читается через PyMuPDF как read-only текстовые страницы; по умолчанию используются первые 5 страниц. В auto-режиме задачи на извлечение и сопоставление по нескольким файлам идут одним корпусом, а exact replacements остаются пофайловыми и применяются только в DOCX-копии в `output\`.

Основной человеческий результат пишется в DOCX с оформлением в стиле audit-отчёта. Табличные сопоставления пишутся в XLSX с тем же тёплым оформлением, что и итоговые audit-таблицы; JSON `values` из ответа модели разворачивается в настоящие колонки.

Для повторяемых реквизитных таблиц TASK поддерживает чистый экспорт по DOCX/XLSX-шаблону из `input\`: выбранный шаблон не отправляется модели как источник, первая строка задаёт колонки, вторая строка задаёт формат значений. Дополнительные чистые файлы пишутся в `output\_doc_tasks\` как `*__doc_task_clean.json`, `*__doc_task_clean.xlsx` и, для DOCX-шаблона, `*__doc_task_clean.docx`.

Файлы для локальных API-ключей:

- `config\api_key_openai.txt`
- `config\api_key_gemini.txt`
- `config\api_key_xai.txt`
- `config\api_key_anthropic.txt`

Также поддерживаются переменные окружения:

- `OPENAI_API_KEY`
- `GEMINI_API_KEY`
- `XAI_API_KEY`
- `ANTHROPIC_API_KEY`

Реальные ключи нельзя коммитить.

Структура репозитория

<code>config/</code>	LLM-настройки, key placeholders, prompt/rules/task Markdown
<code>install/</code>	build, install, verify, release scripts
<code>system_core/</code>	engines, pipeline, renderers, helpers
<code>tests/</code>	unit tests
<code>input/</code>	вход рекурсивного DOCX/PPTX audit pipeline и
<code>DOCX/PPTX/XLSX/PDF document tasks</code>	
<code>output/</code>	пользовательские отчёты и annotated/fixed файлы
<code>logs/</code>	JSON logs, block maps, render maps, audit logs
<code>report/</code>	вспомогательные отчёты и patch-plan нормализации
<code>work/</code>	rendered PDFs, marked OOXML, extracted PDF text
<code>cache/</code>	pipeline cache
<code>runtime/</code>	generated portable Python runtime, не исходники
<code>wheelhouse/</code>	generated wheel cache, не исходники
<code>system_core/powershell/</code>	optional generated portable PowerShell, не исходники
<code>licenses/</code>	generated third-party notices
<code>release/</code>	generated release archives
<code>._runtime/</code>	временные файлы лаунчеров

Сборка и проверка

Основной вход:

```
builder_main.cmd
```

Build-лаунчер отвечает за:

- сборку portable runtime
- offline install
- проверку окружения
- обновление `fzf.exe`
- сбор release archive
- открытие runtime/wheelhouse/release/license папок
- запуск Project EN/RU и Tools

Прямые проверки:

```
runtime\python.exe -m compileall system_core tests
runtime\python.exe -m unittest discover -s tests
install\verify_portable_env.cmd
```

Microsoft Office COM export может падать в restricted/non-interactive сессиях с `0x80070520`. Для настоящей проверки render-map запускайте из обычного интерактивного Windows PowerShell или терминала VS Code.

Tools

Служебный лаунчер:

```
launcher_tools.cmd
```

Tools отвечает за:

- сбор third-party licenses
- collect and deduplicate licenses
- prune stale license folders
- подготовку дерева к GitHub
- открытие папок config/logs/work/output

Подготовка к GitHub

Запуск:


```
cleanup_project.cmd
```

Скрипт удаляет локальное generated/runtime-состояние:

- runtime\
- wheelhouse\
- .venv*
- system_core\powershell\
- system_core\fzf.exe
- caches, outputs, logs, work artifacts
- generated release/license folders
- старые audit launchers и root API-key leftovers

Он оставляет исходники, GitHub-документацию, install-скрипты, tests, configs и пересоздаёт минимальный пустой каркас папок. Root-level API-key leftovers удаляются;

config\api_key_openai.txt, config\api_key_gemini.txt, config\api_key_xai.txt и config\api_key_anthropic.txt нужно проверить и очистить вручную перед публикацией, чтобы реальные ключи не ушли в репозиторий.

Безопасность

Документы могут отправляться внешним LLM-провайдерам. Обработывайте только те документы, которые вам разрешено передавать выбранному провайдеру.

Никогда не коммитьте:

- реальные API-ключи
- приватные исходные документы
- generated logs с чувствительным содержимым
- generated reports с приватным текстом документов
- локальные backup_before_*, QA-render каталоги, PID и Python/test cache